

Ain Shams University
Faculty of Engineering
Mechatronics Engineering Department

Collaborative Robotic Arms Digital-Twin Enabled

Graduation Project Thesis

Submitted in partial fulfillment of the requirements for the
B.Sc. degree in Mechatronics Engineering

Team Members:

Marwan Mahmoud	Mohamed Tarek
Mariam El Sebaey	Mohamed Alaa
Omar Magdy	Ingy Ahmed
Yehia Raouf	Kareem Saleh

Supervisors:

Dr. Mohamed Omar
Dr. Shady Ahmed
Eng. Dina Zakaria

Academic Year: 2025–2026
Cairo, Egypt

Declaration

We/I hereby certify that this Project submitted as part of our/my partial fulfillment of BSc in Mechatronics Engineering is entirely our/my own work, that we/I have exercised reasonable care to ensure its originality, and does not to the best of our/my knowledge breach any copyrighted materials, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our/my work.

Signed by:

Name	Signature
Ingy Ahmed Sherif	
Kareem Saleh Hassan	
Mariam Mohamed El Sebaey	
Marwan Mahmoud Ali	
Mohamed Alaa Abdelkareem	
Mohamed Tarek Abdelwahab	
Omar Magdy Abdelsattar	
Yahia Mohamed Raouf	

Date: January 29, 2026

Acknowledgment

The team would like to thank our supervisors,

Abstract

This thesis presents a Digital-Twin-enabled collaborative robotic assembly system integrating an ABB industrial manipulator and an AR4 robotic arm to execute efficient brick assembly through coordinated perception, grasping, handover, and placement. The proposed system combines global scene understanding from an Intel RealSense camera with local “eye-in-hand” sensing using an ESP32-CAM mounted on the AR4, enabling both coarse positioning and fine alignment during grasping via Image-Based Visual Servoing (IBVS). A structured state machine governs end-to-end task allocation and execution, selecting between direct assembly and cross-side handover based on brick and robot allocation logic derived from GUI/database input and global perception.

To achieve accurate interaction in a shared workspace, calibration is treated as a core requirement. AR4 calibration is performed by collecting data at multiple X–Y locations at two height levels (0 mm and 50 mm) and fitting mapping functions using multiple approaches (Radial Basis Function (RBF), linear, and polynomial). Polynomial regression demonstrated superior performance and was validated using training/testing splits. Similarly, global camera calibration based on polynomial regression is applied using samples across varying X, Y, and Z coordinates to enable reliable estimation of brick world pose across the entire table. Perception is implemented through YOLOv8 segmentation for brick and feature detection, while grasp selection is addressed using a transformer-based encoder–decoder trained with both successful grasps (True Grasps) and failure/avoidance cases (False Grasps), enabling negative reinforcement to reduce unstable grasp predictions. During fine control, homography is used to map feature points between the RealSense global view and the ESP32-CAM local view, simplifying tracking and improving approach robustness.

Experimental results demonstrate improved positional accuracy using polynomial calibration, effective brick detection performance using segmentation, increased grasp stability using the True/False labeling strategy, and robust end-to-end execution under a state-machine framework. The thesis concludes with limitations and future directions, including improved domain adaptation, force-aware handovers, and enhanced Digital Twin synchronization for higher fidelity planning and monitoring.

Contents

1	Introduction	7
1.1	Background and Motivation	7
1.2	Problem Statement	7
1.3	Project Objectives	8
1.4	Project Scope	8
1.5	Key Contributions	9
1.6	Thesis Organization	9
2	System Design and Methodology	10
2.1	Cartesian Calibration of the AR4 Collaborative Robot	10
2.1.1	Experimental Setup	10
2.1.2	Cartesian Error Characterization	10
2.1.3	Calibration Method Selection	11
2.1.4	Polynomial Calibration Model	11
2.1.5	Calibration Results	11
2.1.6	Error Surface Analysis	12
2.1.7	Quantitative Evaluation	14
2.2	Assembly Supervisor and State Machine Control	14
2.2.1	Overview	14
2.2.2	Supervisor Architecture and Responsibilities	14
2.2.3	ROS 2 Communication: Services and Actions	15
2.2.4	Transform (TF2) Management and Coordinate Frames	17
2.2.5	Detailed State Machine Analysis	18



2.2.6	Error Handling and Recovery	26
2.2.7	Concurrency and Threading Model	27
2.2.8	State Machine Timing and Scheduling	28
2.2.9	Hardware and Sensor Integration	28
2.2.10	Data Flow and Message Passing	29
2.2.11	Debugging and Logging	30
2.2.12	Summary	31

Chapter 1

Introduction

1.1 Background and Motivation

Collaborative robotic systems are increasingly adopted in modern automation to improve throughput, flexibility, and safety in assembly workflows. While a single industrial robot can achieve high repeatability, multi-robot collaboration enables task decomposition, parallelism, and improved workspace coverage.

Despite these advantages, multi-robot collaboration introduces substantial technical challenges. These include:

- Accurate calibration between sensing systems and robotic manipulators.
- Reliable perception under dynamic and uncertain real-world conditions.
- Robust grasp planning to avoid object slippage or collision.
- Stable closed-loop control for fine alignment during assembly.
- Coordination of system-level logic to manage roles, task handovers, and execution uncertainties.

This project addresses these challenges by implementing a two-robot collaborative assembly system, integrating an ABB industrial robot and an AR4 research robot to perform brick assembly tasks. The system leverages Digital Twin technology to provide visualization, monitoring, and structured integration of perception, planning, and execution modules.

1.2 Problem Statement

The primary problem addressed in this project is the design and implementation of a robust, two-robot collaborative assembly system capable of:

1. Detecting and localizing bricks within a shared workspace using global perception.
2. Selecting stable grasps and avoiding unstable configurations via learned models.
3. Executing safe and accurate grasping using hybrid control strategies (point control combined with image-based visual servoing).

4. Performing reliable handovers from the AR4 to the ABB robot when bricks need to cross workspace sides.
5. Maintaining a consistent world coordinate system across the workspace through calibration of cameras and robots.
6. Coordinating the full workflow via a state machine with well-defined transitions and recovery logic.

1.3 Project Objectives

The objectives of this thesis are summarized as follows:

- Develop a state machine to govern setup, task allocation, grasping, handover, and assembly operations reliably.
- Implement robust object detection using YOLOv8 segmentation for bricks and workspace features.
- Implement grasp detection using a transformer-based encoder-decoder trained with stable and unstable grasps to improve reliability.
- Implement fine alignment for AR4 grasping using image-based visual servoing aided by homography mapping between global and local camera views.
- Achieve accurate AR4 position correction using data-driven calibration techniques (tested: radial basis function, linear, polynomial; best performance: polynomial regression).
- Calibrate the global camera-to-world mapping for reliable brick pose estimation across the entire workspace.
- Develop a Digital Twin-enabled architecture integrating perception, decision logic, and robot execution.

1.4 Project Scope

This thesis focuses on the perception-to-action integration for collaborative assembly using two robotic arms and two camera systems (global and eye-in-hand). The scope includes:

- System calibration and camera-to-robot alignment.
- Object detection and grasp pose estimation.
- Visual servoing and hybrid control for precise manipulation.
- Handover logic between robots.
- Digital Twin integration for monitoring and validation.

The thesis does **not** include:

- Detailed mechanical design of the robots.
- Proprietary low-level controller programming of the ABB robot.
- Safety certification processes beyond safe experimental operation.

1.5 Key Contributions

The main contributions of this project are:

1. Implementation of a Digital Twin-enabled multi-robot assembly system for visualization, validation, and monitoring.
2. Development of a polynomial-based calibration method for precise AR4 positioning and global workspace mapping.
3. Integration of YOLOv8-based object detection for brick and feature localization.
4. Implementation of a transformer-based grasp detection network for improved grasp stability.
5. Hybrid control strategy combining point control and image-based visual servoing for fine alignment.
6. Reliable handover and task coordination using a state machine with recovery mechanisms.

1.6 Thesis Organization

This thesis is organized as follows:

- **Chapter 2** reviews literature on collaborative robotics, multi-robot handover, visual servoing, grasp detection, and Digital Twin integration.
- **Chapter 3** describes the system architecture, including calibration, perception, grasping, control, and workflow management.
- **Chapter 4** presents the experimental implementation, evaluation of calibration accuracy, perception performance, grasp prediction quality, and workflow reliability.
- **Chapter 5** concludes the thesis, summarizes key findings, and discusses potential future enhancements.

Chapter 2

System Design and Methodology

2.1 Cartesian Calibration of the AR4 Collaborative Robot

Accurate positioning is critical in collaborative robotic applications, particularly when multiple manipulators operate within a shared workspace. During initial experimental trials, the AR4 robotic arm exhibited noticeable Cartesian positioning inaccuracies when executing commanded end-effector positions relative to the ABB robot world coordinate frame. These inaccuracies motivated the implementation of a dedicated Cartesian calibration procedure to improve the absolute positioning accuracy of the AR4 robot.

2.1.1 Experimental Setup

The calibration experiment was conducted using a pointed end-effector mounted on the AR4 robot to enable precise position referencing. A calibrated measurement grid with millimeter resolution was placed on the worktable. The grid origin was aligned with the ABB robot world frame origin (0,0,0), ensuring a common global reference for both manipulators.

The AR4 robot was commanded to move to a predefined set of Cartesian positions distributed over the XY plane at multiple height levels along the Z-axis. At each commanded position ($x_{cmd}, y_{cmd}, z_{cmd}$), the actual end-effector position ($x_{meas}, y_{meas}, z_{meas}$) was manually recorded using the grid.

2.1.2 Cartesian Error Characterization

The raw Cartesian positioning errors were computed as:

$$\begin{aligned} e_x &= x_{meas} - x_{cmd} \\ e_y &= y_{meas} - y_{cmd} \\ e_z &= z_{meas} - z_{cmd} \end{aligned} \tag{2.1}$$

Initial analysis revealed systematic and spatially correlated errors across the workspace, indicating that the inaccuracies were not purely random but instead dependent on the commanded Cartesian position.

2.1.3 Calibration Method Selection

Several calibration approaches were evaluated prior to selecting the final method. Table 2.1 summarizes the comparison between candidate techniques. The calibration dataset was divided into independent training and testing subsets to ensure that the reported performance reflects generalization rather than overfitting to the calibration points.

Method	Accuracy	Extrapolation	Data Requirements	Real-Time
Lookup Table	Medium	None	High	High
RBF Network	High	Poor	Very High	Medium
Spline Fitting	High	Limited	Medium	High
Polynomial Regression	High	Good (Low-Order)	Medium	High

Table 2.1: Comparison of Cartesian calibration methods for robotic manipulators

Based on this comparison, the polynomial calibration was selected as a balanced solution, offering a favorable trade-off between accuracy, robustness, and implementation simplicity, making it particularly suitable for real-time correction in collaborative robotic systems.

2.1.4 Polynomial Calibration Model

A second-order polynomial regression model was used to approximate the Cartesian error as a function of the commanded XY position:

$$e(x, y) = a_1x + a_2y + a_3x^2 + a_4y^2 + a_5xy + a_6 \quad (2.2)$$

The estimated polynomial error models were used to correct the commanded positions prior to execution:

$$\begin{aligned} x_{corr} &= x_{cmd} + \hat{e}_x(x, y) \\ y_{corr} &= y_{cmd} + \hat{e}_y(x, y) \\ z_{corr} &= z_{cmd} + \hat{e}_z(x, y) \end{aligned} \quad (2.3)$$

This correction strategy enables feedforward compensation of systematic positioning errors without modifying the internal robot controller.

2.1.5 Calibration Results

Based on the experimental results, all evaluated calibration approaches significantly reduced the Cartesian positioning error when compared to the uncalibrated model. Among the tested methods, the linear model exhibited the highest residual error due to its inability to capture nonlinear kinematic and geometric effects. Radial Basis Function (RBF) networks achieved the lowest mean positioning error, demonstrating superior interpolation accuracy within the calibrated workspace. However, their locally adaptive nature resulted in less predictable behavior near the workspace boundaries and outside the

training region.

The second-degree polynomial model produced slightly higher errors than the RBF-based approach, yet these errors remained within acceptable limits for the intended application. In contrast to RBF and spline-based methods, the polynomial model exhibited smoother and more stable behavior near workspace boundaries, while maintaining low computational cost and consistent real-time performance.

Figure 2.1 illustrates a comparison between the commanded, measured, and polynomial-corrected end-effector positions in the XY plane. The deviation between the commanded and measured positions highlights the presence of systematic Cartesian positioning errors in the AR4 robot, while the corrected positions demonstrate the effectiveness of the polynomial calibration in compensating for these errors.

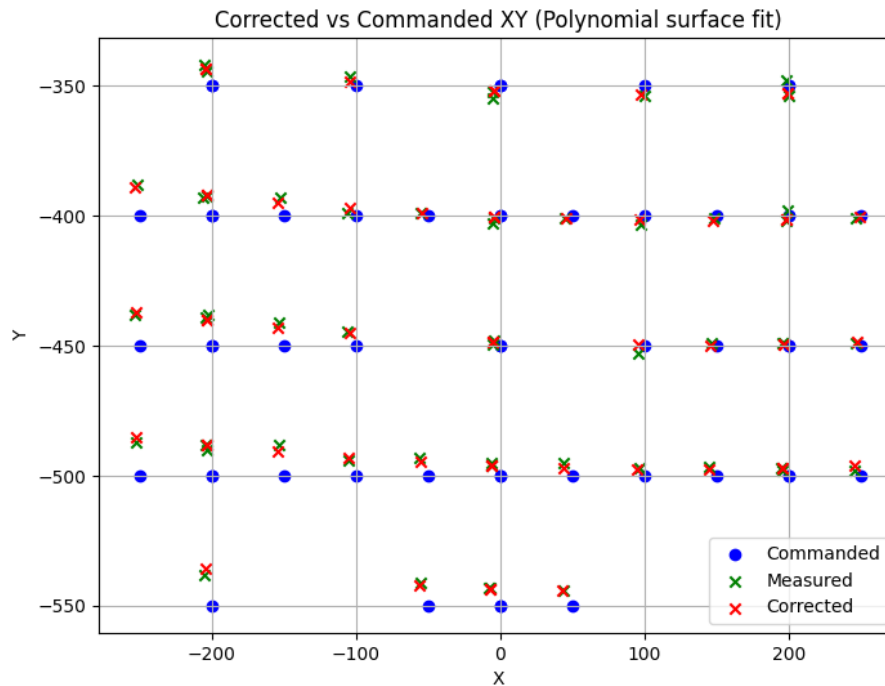


Figure 2.1: XY-plane commanded, measured, and corrected positions.

2.1.6 Error Surface Analysis

To further analyze the spatial behavior of the calibration model, three-dimensional polynomial error surfaces were generated for each Cartesian axis, which are shown in Figure 2.2

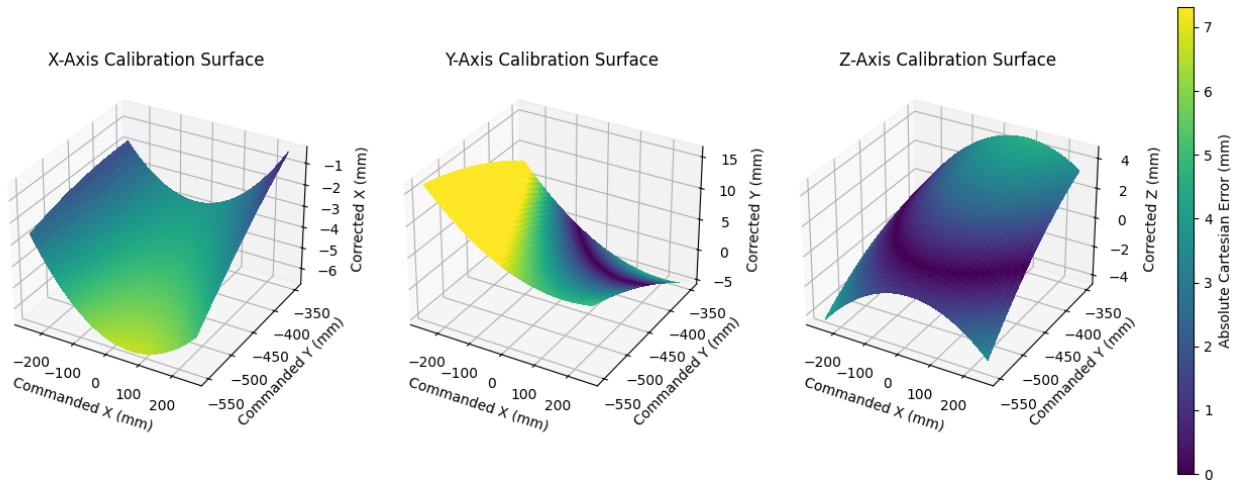


Figure 2.2: Calibration surfaces for the X, Y, and Z axes with error magnitude.

Figure 2.3 presents a comparison between the uncalibrated and calibrated surfaces. The comparison demonstrates that the calibration model effectively smooths and compensates for the dominant systematic error components, resulting in a more uniform and stable error distribution across the workspace.

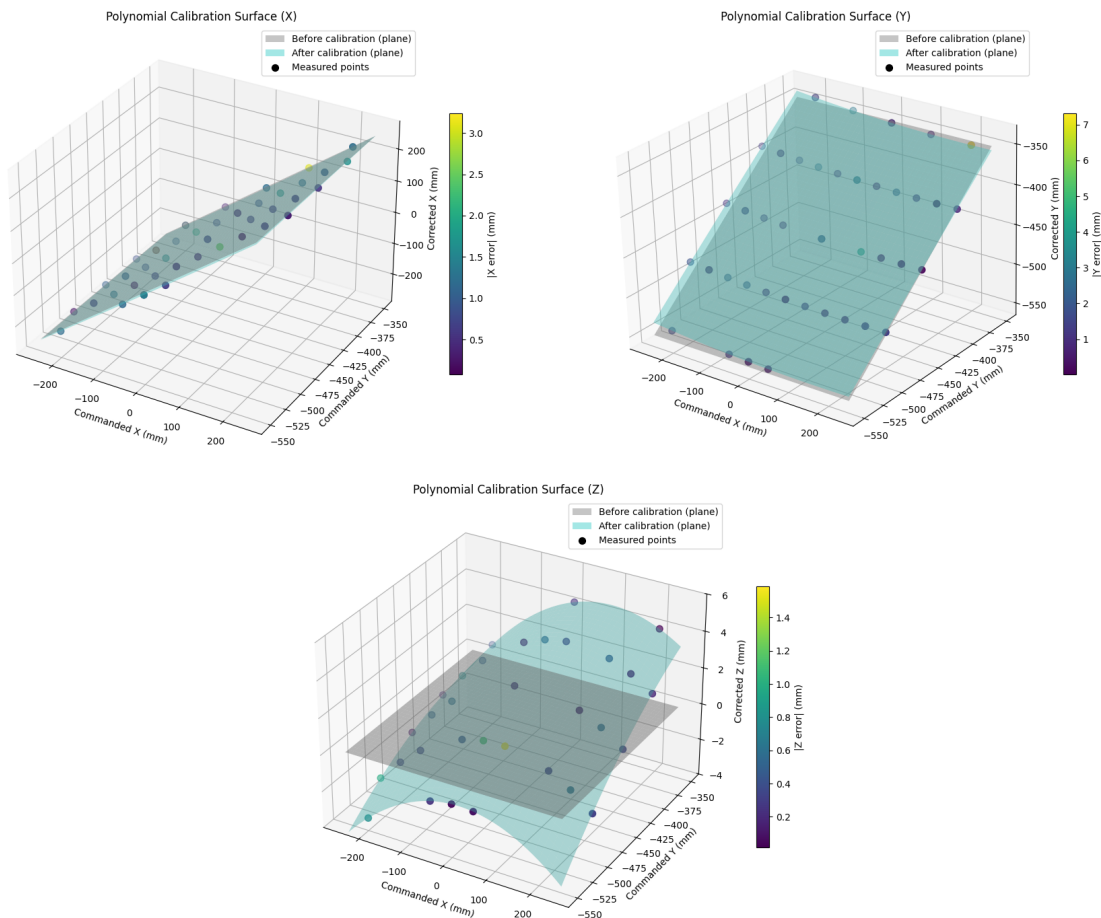


Figure 2.3: Comparison of uncalibrated and calibrated surfaces.

2.1.7 Quantitative Evaluation

The effectiveness of the calibration was quantitatively evaluated using the mean absolute error (MAE) before and after correction for each Cartesian axis, computed on the independent test dataset.

Table 2.2: Mean Absolute Cartesian Errors Before and After Calibration

Axis	MAE Before (mm)	MAE After (mm)
X	3.96	0.94
Y	5.13	1.32
Z	1.94	0.46

The quantitative results in Table 2.2 are consistent with the smooth and continuous behavior observed in the polynomial error surfaces shown in Figure 2.2, confirming the effectiveness and stability of the proposed calibration approach.

2.2 Assembly Supervisor and State Machine Control

2.2.1 Overview

The Assembly Supervisor is the central orchestrator of the collaborative robotic assembly system. It implements a robust state machine that governs the complete workflow from task initialization through brick detection, grasp planning, robot coordination, and final assembly placement. This section provides a comprehensive analysis of the supervisor's architecture, state transitions, service and action interactions, and error handling mechanisms that ensure reliable system operation in a collaborative multi-robot environment.

2.2.2 Supervisor Architecture and Responsibilities

High-Level Architecture

The Assembly Supervisor operates as a ROS 2 node with the following core responsibilities:

- **Task Coordination:** Retrieve assembly plans from the GUI and manage task allocation across the two robotic arms.
- **Perception Integration:** Coordinate camera-based brick detection and transform detected poses into the appropriate reference frames.
- **Grasp Inference:** Query the grasp pipeline to select stable grasps for each brick.
- **Motion Execution:** Issue action goals to the AR4 and ABB control servers to execute pick, place, and handover operations.

- **Coordination Logic:** Decide between direct assembly and handover strategies based on brick location and arm allocation.
- **Reference Frame Management:** Transform all detected poses and grasp points into the appropriate coordinate frames using TF2 (ROS 2 Transform Library).

Execution Model and Threading

The supervisor employs a multi-threaded executor with a `ReentrantCallbackGroup` to permit concurrent handling of asynchronous ROS 2 callbacks. This design allows:

- Parallel service queries and action goal submissions without blocking the main state machine loop.
- Safe re-entrant locking when multiple callbacks access shared state variables.
- Non-blocking timer-based state machine updates at a controlled frequency (initially 1.0 Hz, reduced to 0.1 Hz during active task execution).

Figure 2.4: Assembly Supervisor Architecture: Clients, Callbacks, and State Machine

2.2.3 ROS 2 Communication: Services and Actions

Service Clients

Services provide synchronous request-response communication for deterministic operations that do not require real-time feedback:

1. `get_assembly_plan` (Service)

- **Purpose:** Retrieve the assembly plan (ordered list of bricks) from the GUI node.
- **Interface:** `supervisor_package/srv/GetAssemblyPlan`
- **Response Fields:** Plan array containing brick ID, location, pickup pose, and place pose.
- **Invocation:** Called once during INIT state to bootstrap the assembly sequence.

2. `detect_bricks` (Service)

- **Purpose:** Trigger YOLOv8 segmentation and brick pose estimation from the global RealSense camera.
- **Interface:** `dual_arms_msgs/srv/DetectBricks`
- **Response Fields:** Array of detected bricks with world poses, plus a computed handover pose (neutral zone between robots).

- **Invocation:** Called during DETECT state to refresh spatial understanding after assembly plan receipt.

3. grasp/get_grasp_point (Service)

- **Purpose:** Query the transformer-based grasp inference model for the optimal grasp candidate for a specific brick.
- **Interface:** `dual_arms_msgs/srv/GetGrasp`
- **Request Fields:** Brick index (converted to string).
- **Response Fields:** A `GraspPoint` message containing pose, quality metric, and success indicator.
- **Invocation:** Called for each brick during GRASP_PIPELINE state before attempting to grasp.

Action Clients

Actions provide asynchronous, feedback-capable communication for long-running operations. The supervisor uses action clients to submit motion goals and wait for completion:

1. ar4_point_control (Action)

- **Purpose:** Send point-to-point motion commands to the AR4 arm (approach, grasp, release, placement).
- **Interface:** `supervisor_package/action/MoveToPose`
- **Goal Fields:** Target pose, strategy (APPROACH_OFFSET, GRASP, PLACE, GOTO_HANDOVER, RELEASE).
- **Used in States:** EXECUTE_AR4_DIRECT, AR4_PICK_FOR_HANDOVER, AR4_PLACE_ON_GRID, HANDOVER_EXECUTION.

2. ar4_visual_servo (Action)

- **Purpose:** Execute Image-Based Visual Servoing (IBVS) feedback control via the ESP32-CAM.
- **Interface:** `supervisor_package/action/AlignToTarget`
- **Goal Fields:** Object ID (brick type), homography mapping parameters.
- **Behavior:** Iteratively refines AR4 position to align the brick in the ESP32 camera frame, halting once pixel error is below threshold.
- **Used in States:** EXECUTE_AR4_DIRECT, AR4_PICK_FOR_HANDOVER.

3. abb_control (Action)

- **Purpose:** Issue high-level task commands to the ABB robot (pick from detected pose, place, pick from handover).
- **Interface:** `dual_arms_msgs/action/ExecuteTask`
- **Goal Fields:** Task type (PICK, PLACE, PICK_FROM_HANDOVER), target pose.
- **Behavior:** The ABB controller internally manages gripper open/close, motion planning, and collision avoidance.
- **Used in States:** EXECUTE_ABB_PICK, EXECUTE_ABB_PLACE, HANDOVER_EXECUTION.

Communication Flow Diagram

Figure 2.5: Supervisor Service and Action Communication Flow

2.2.4 Transform (TF2) Management and Coordinate Frames

Coordinate Frame Hierarchy

The system employs multiple coordinate frames, necessitating careful transformation management:

- **base_link:** The ABB robot base (designated as the primary world reference frame).
- **camera_color_optical_frame:** The RealSense camera's optical center.
- **ar4_base:** The AR4 robot's base frame.
- **ar4_tool:** The AR4 end-effector (gripper) frame.
- **esp32_cam_optical:** The ESP32-CAM's optical frame (eye-in-hand).
- **table:** A virtual frame representing the assembly table surface.

Transform Pipeline

The supervisor uses TF2 to establish and maintain transformations:

1. Brick Detection to World Frame:

- Input: Brick pose in `camera_color_optical_frame` from YOLOv8 segmentation.
- Transformation: `lookup_transform('base_link', 'camera_color_optical_frame')`
- Output: Brick pose in `base_link` (ABB world frame) for unified task planning.

2. Grasp Point Transformation:

- The grasp pipeline returns a `GraspPoint` in the camera frame.
- The supervisor applies the same camera-to-base transformation.
- Additional adjustments are made: Z-coordinate clamped to safety height (0.22 m), orientation components reordered to match AR4 conventions.

3. Handover Pose Transformation:

- The global detection service computes a handover pose (midpoint between the two robots).
- The supervisor transforms this pose to `base_link` for consistency.

Transform Implementation Details

The supervisor implements transform lookups with error handling. A typical lookup operation:

```
def transform_pose_to_abb(self, input_pose):
    try:
        t = self.tf_buffer.lookup_transform(
            'base_link',
            'camera_color_optical_frame',
            rclpy.time.Time())
        transformed_pose = tf2_geometry_msgs.do_transform_pose(
            input_pose, t)
        return transformed_pose
    except TransformException as ex:
        self.get_logger().error(f'TF2 Error: {ex}')
        return input_pose # Fallback
```

This implementation:

- Uses `rclpy.time.Time()` to request the latest available transform.
- Falls back to the original pose if transformation fails (avoiding system crash).
- Logs transformation errors for debugging.

Figure 2.6: Coordinate Frame Hierarchy and Transform Chain

2.2.5 Detailed State Machine Analysis

State Machine Overview

The Assembly Supervisor implements a 12-state finite state machine that governs the complete assembly workflow. States are organized into five logical stages:

1. **Stage 1 - Initialization (INIT):** Retrieve assembly plan and validate input.
2. **Stage 2 - Perception (DETECT):** Obtain global brick poses and compute handover zone.
3. **Stage 3 - Task Allocation (PROCESS_NEXT, GRASP_PIPELINE):** Select next brick and compute grasp.
4. **Stage 4 - Execution (Multiple states):** Execute grasping, handover, or placement based on allocation.
5. **Stage 5 - Termination (DONE):** All tasks complete; system idle.

Figure 2.7: Complete State Machine Diagram with Transitions

State 1: INIT (Initialization)

Purpose Retrieve the assembly plan from the GUI and bootstrap the state machine. This state validates that a plan is available before proceeding to perception.

Execution Logic

1. Check if the `gui_client` service is available using `wait_for_service(timeout_sec=1.0)`.
2. If unavailable, log a message and reschedule the state machine loop after a 2-second delay.
3. If available, submit a `GetAssemblyPlan` service request (empty request body).
4. Validate the response: Check that `result.plan` is not `None` and contains at least one brick.
5. On valid plan receipt, store plan in `self.assembly_queue` and transition to DETECT.
6. On invalid plan, log a warning and reschedule after 2 seconds (remain in INIT).

Error Handling

- **Service Not Ready:** Reschedule and wait. This handles GUI node startup delays.
- **Empty Plan:** Log and retry. This distinguishes between “service not ready” and “no plan available”.
- **Network Failure:** Timeout on `wait_for_service` is caught; state remains INIT.

Transition

- Success: INIT → DETECT
- Retry: INIT → INIT (after 2 seconds)

State 2: DETECT (Global Perception)

Purpose Invoke YOLOv8 segmentation and pose estimation to obtain brick positions in the global camera frame. Transform all poses to the ABB base frame.

Execution Logic

1. Check service availability: `self.camera_client.wait_for_service(timeout_sec=1.0)`.
2. If unavailable, log error and reschedule.
3. Submit a `DetectBricks` service request.
4. Receive response containing:
 - `result.bricks[*]`: Array of detected bricks with pose in camera frame.
 - `result.handover_pose`: Computed neutral zone pose in camera frame.
5. For each brick, apply transform: `brick.pose = self.transform_pose_to_abb(brick.pose)`.
6. Similarly, transform handover pose: `self.handover_pose = self.transform_pose_to_abb(result.handover_pose)`.
7. Store transformed bricks in `self.detected_bricks`.
8. Transition to `PROCESS_NEXT`.

Pose Validation After transformation, poses are validated:

- Z-coordinate should be non-negative (on or above table).
- X, Y coordinates should be within workspace bounds.
- Orientation quaternion should be normalized.

Transition

- Success: `DETECT` → `PROCESS_NEXT`

State 3: `PROCESS_NEXT` (Task Dequeue)

Purpose Remove the next brick from the assembly queue and initiate grasp planning. This state acts as a checkpoint for per-brick initialization.

Execution Logic

1. Check if `self.assembly_queue` is empty.
2. If empty: transition to `DONE` and exit.
3. If non-empty: pop the first element and assign to `self.current_brick`.
4. Transition to `GRASP_PIPELINE`.

Transition

- Bricks remaining: PROCESS_NEXT → GRASP_PIPELINE
- No bricks: PROCESS_NEXT → DONE

State 4: GRASP_PIPELINE (Grasp Inference)

Purpose Query the transformer-based grasp inference model to obtain a stable grasp candidate for the current brick. Apply pose transformation and additional safety adjustments.

Execution Logic

1. Check service availability: `self.grasp_pipeline_client.wait_for_service(timeout_sec=2.0)`.
2. If unavailable, log error, transition to PROCESS_NEXT (skip brick), and return.
3. Create a `GetGrasp.Request()` with `brick_index = str(self.current_brick.id)`.
4. Submit request asynchronously and await response.
5. Check `grasp_result.success` flag:
 - If True: Extract `grasp_result.grasp_point` (a `GraspPoint` message).
 - If False: Log error, transition to PROCESS_NEXT, skip brick.
6. Apply coordinate frame transformation.
7. Apply safety adjustments (clamp Z-coordinate, reorder orientation).
8. Store transformed grasp in `self.current_grasp_point`.
9. Determine arm allocation based on `self.current_brick.location`.

Grasp Point Structure The `GraspPoint` message contains:

<code>geometry_msgs/Pose pose</code>	<code># Grasp position and orientation</code>
<code>float32 quality</code>	<code># Quality metric [0, 1]</code>
<code>float32 width</code>	<code># Finger opening distance (meters)</code>
<code>float32 approach_angle</code>	<code># Recommended approach angle (radians)</code>

Arm Allocation Logic Based on brick location from the assembly plan:

- `location == 1`: Brick on ABB's side → EXECUTE_ABB_PICK
- `location == 2`: Brick requires handover → HANDOVER_SEQUENCE
- Other: Brick on AR4's side → EXECUTE_AR4_DIRECT

Transition

- Grasp success, ABB-side brick: GRASP_PIPELINE → EXECUTE_ABB_PICK
- Grasp success, AR4-side brick: GRASP_PIPELINE → EXECUTE_AR4_DIRECT
- Grasp success, handover-required: GRASP_PIPELINE → HANDOVER_SEQUENCE
- Grasp failure: GRASP_PIPELINE → PROCESS_NEXT

State 5: EXECUTE_AR4_DIRECT (AR4 Pick and Place)

Purpose Execute a complete pick-and-place cycle using the AR4 robot for bricks allocated to the AR4's workspace.

Sub-Step A: Approach with Offset

1. Create a `MoveToPose` action goal with strategy "APPROACH_OFFSET".
2. The AR4 controller offsets in the approach direction to maintain safe clearance.
3. Await completion.

Sub-Step B: Visual Servoing (IBVS)

1. Create an `AlignToTarget` action goal with object ID and homography parameters.
2. The visual servoing action server:
 - Captures images from the ESP32-CAM.
 - Detects the brick and computes image-space error.
 - Uses IBVS feedback law to generate AR4 velocity commands.
 - Continues until error is below threshold.
3. Await completion.

Sub-Step C: Grasp and Retract

1. Create a `MoveToPose` action goal with strategy "GRASP".
2. The AR4 controller executes final approach and closes the gripper.
3. Await completion.

Transition

- Success: EXECUTE_AR4_DIRECT → AR4_PLACE_ON_GRID

State 6: AR4_PLACE_ON_GRID (AR4 Placement)

Purpose Move the grasped brick to the assembly grid location specified by the plan.

Execution Logic

1. Create a `MoveToPose` action goal with strategy "PLACE".
2. Submit to AR4 point control action server.
3. The AR4 controller moves and opens the gripper.
4. Await completion.
5. Transition to PROCESS_NEXT.

Transition

- Success: AR4_PLACE_ON_GRID → PROCESS_NEXT

State 7: EXECUTE_ABB_PICK (ABB Grasping)

Purpose Execute pick operation using the ABB robot for bricks on the ABB's side of the workspace.

Execution Logic

1. Create an `ExecuteTask` action goal with task type "PICK".
2. Submit to ABB control action server.
3. The ABB controller plans and executes the pick sequence.
4. Await completion.
5. Transition to EXECUTE_ABB_PLACE.

Transition

- Success: EXECUTE_ABB_PICK → EXECUTE_ABB_PLACE

State 8: EXECUTE_ABB_PLACE (ABB Placement)

Purpose Place the grasped brick at its designated location on the assembly grid.

Execution Logic

1. Create an `ExecuteTask` action goal with task type “PLACE”.
2. Apply safety adjustment: Set Z-coordinate to 0.24 m.
3. Submit to ABB control action server.
4. Await completion.
5. Transition to `PROCESS_NEXT`.

Transition

- Success: `EXECUTE_ABB_PLACE` → `PROCESS_NEXT`

State 9: HANDOVER_SEQUENCE (Handover Initiation)

Purpose Transition to handover workflow when a brick must be transferred from AR4 to ABB.

Execution Logic

1. This state is a pass-through (no action).
2. Immediately transition to `AR4_PICK_FOR_HANDOVER`.

Transition

- `HANDOVER_SEQUENCE` → `AR4_PICK_FOR_HANDOVER`

State 10: AR4_PICK_FOR_HANDOVER (AR4 Grasping for Transfer)

Purpose Execute the AR4 grasping sequence in preparation for handover.

Execution Logic See Sub-Steps A, B, C under State 5 (`EXECUTE_AR4_DIRECT`).

Transition

- Success: AR4_PICK_FOR_HANOVER → HANOVER_EXECUTION

State 11: HANOVER_EXECUTION (Coordinated Transfer)

Purpose Execute synchronized handover between the AR4 and ABB robots.

Step 1: AR4 Move to Handover Zone

1. Create a `MoveToPose` action goal with strategy “GOTO_HANOVER”.
2. Submit to AR4 point control action server.
3. Await completion.

Step 2: ABB Approach and Grasp

1. Create an `ExecuteTask` action goal with task type “PICK_FROM_HANOVER”.
2. Submit to ABB control action server.
3. The ABB controller approaches and closes gripper.
4. Await completion.

Step 3: AR4 Release

1. Create a `MoveToPose` action goal with strategy “RELEASE”.
2. Submit to AR4 point control action server.
3. The AR4 controller opens gripper and retracts.
4. Await completion.

Transition

- Success: HANOVER_EXECUTION → PROCESS_NEXT

Handover Synchronization Considerations

- **Sequential Approach:** Strict sequential steps ensure clear state and avoid conflicts.
- **Force Feedback (Future):** Force/torque sensors could enable simultaneous dual-robot grasp confirmation.
- **Fallback Strategy:** If ABB grasp fails, AR4 does not release; system can retry.

State 12: DONE (Completion)

Purpose Signal that all tasks are completed and the system is idle.

Execution Logic

1. Log completion message.
2. Do not reschedule the state machine loop timer.
3. System waits for external INIT signal or restart.

Transition

- DONE → (Await external signal)

2.2.6 Error Handling and Recovery

Service Availability Checking

The supervisor checks service availability before submitting requests. If unavailable:

- Log a diagnostic message.
- Reschedule the state machine loop after a delay (2 seconds).
- Remain in current state, retrying the same action.

Action Goal Failure

When an action goal is rejected:

- Check the goal handle status.
- Log an error and return False.
- The calling state decides on recovery (retry or skip).

Transform Lookup Failures

If a TF2 transform is unavailable:

- Catch `TransformException`.
- Log the exception.
- Return the original pose as fallback.

Grasp Pipeline Failures

If the grasp service returns `success=False`:

- Log the failure.
- Transition to `PROCESS_NEXT` (skip the brick).

Robot Timeout

If an action goal does not complete within a reasonable time:

- Cancel the goal and log the failure.
- Transition to `PROCESS_NEXT`.
- Proceed with the next brick.

2.2.7 Concurrency and Threading Model

ReentrantCallbackGroup

The supervisor uses `ReentrantCallbackGroup` to allow concurrent callback execution:

- Asynchronous service responses processed during state machine operation.
- Non-blocking behavior for long-running actions.

MultiThreadedExecutor

The supervisor node uses `MultiThreadedExecutor`:

- Allocates thread pool for callback handling.

- Async/await patterns execute without blocking.
- Safe re-entrant locking for shared state.

Potential Race Conditions

- **assembly_queue modification:** Mitigated by single-threaded state machine logic.
- **current_brick and grasp data:** Protected by sequential state transitions.
- **detected_bricks refresh:** Snapshot consistency assumed.

2.2.8 State Machine Timing and Scheduling

Timer Management

The supervisor uses `rclpy.time.Timer` for periodic state machine invocations:

- **Initialization:** Timer created at 1.0 Hz frequency.
- **Dynamic Frequency:** Reset to 0.1 Hz during heavy computation to reduce CPU overhead.
- **Cancellation:** Timer canceled when state machine completes or encounters blocking actions.

Async/Await Pattern

The state machine loop is asynchronous, enabling:

- Non-blocking waiting for service responses.
- Clean exception handling and state transitions.
- Delayed task scheduling and timer rescheduling.

2.2.9 Hardware and Sensor Integration

Robotic Arms

ABB Robot

- **Model:** IRB 1200-5/90 (industrial manipulator).
- **Workspace:** Covers placement side of assembly table.
- **Gripper:** Two-finger parallel gripper.

- **Communication:** ROS 2 action server at `abb_control`.
- **Payload Capacity:** Sufficient for single brick grasping.

AR4 Robot

- **Model:** Annin Robotics AR4 (4-DOF SCARA-style arm).
- **Workspace:** Covers grasping/pickup side.
- **Gripper:** Two-finger gripper controlled via ESP32.
- **Eye-in-Hand Camera:** ESP32-CAM for local perception.
- **Communication:** ROS 2 action servers at `ar4_point_control` and `ar4_visual_servo`.
- **Calibration:** Position correction via polynomial regression (Section 2.1).

Cameras and Perception

Intel RealSense D435i (Global Perception)

- **Placement:** Fixed above assembly table covering full workspace.
- **Output:** RGB images for YOLOv8 segmentation.
- **Calibration:** Camera-to-ABB-base transformation in TF2.
- **Update Rate:** 30 Hz.

ESP32-CAM (Eye-in-Hand Perception)

- **Placement:** Mounted on AR4 end-effector.
- **Output:** Low-resolution RGB for feature detection and visual servoing.
- **Calibration:** Homography mapping from RealSense to ESP32 view.
- **Update Rate:** 5–10 Hz.
- **Latency:** 100–200 ms (mitigated by low-gain feedback control).

2.2.10 Data Flow and Message Passing

Brick Representation

The system represents bricks with the following structure:

```
dual_arms_msgs/Brick:
  int32 id           # Unique identifier
  int32 type         # Brick class
  int32 location     # 1=ABB side, 2=Handover, other=AR4
  geometry_msgs/Pose pickup_pose
  geometry_msgs/Pose place_pose
```

Grasp Point Representation

Grasp candidates are represented as:

```
dual_arms_msgs/GraspPoint:
  geometry_msgs/Pose pose      # Position and orientation
  float32 quality             # Metric [0, 1]
  float32 width               # Finger opening [meters]
  float32 approach_angle     # Approach direction
```

2.2.11 Debugging and Logging

Log Levels

The supervisor uses ROS 2 logging with different verbosity levels:

- **INFO:** State transitions, service calls, task completion.
- **WARN:** Empty plans, service delays, non-critical failures.
- **ERROR:** Service unavailability, transform failures, action rejections.

State Machine Monitoring

Current logging provides visibility into:

- State transitions.
- Task counts.
- Brick-specific progress.
- Grasp quality metrics.

2.2.12 Summary

The Assembly Supervisor implements a robust, multi-threaded state machine that orchestrates a collaborative two-robot assembly system. Key features include:

1. **Modular Service/Action Architecture:** Clean separation between perception, execution, and decision logic.
2. **Transform Management:** Consistent use of TF2 for coordinate frame conversions.
3. **Comprehensive State Machine:** 12 states covering all operational phases with clear error recovery paths.
4. **Concurrency Support:** MultiThreadedExecutor enables responsive, non-blocking operation.
5. **Hardware Integration:** Direct control of two heterogeneous arms with different control models.
6. **Error Resilience:** Timeouts, service failures, and inference failures handled gracefully.

The supervisor's structured approach provides a foundation for reliable, repeatable collaborative assembly operations and facilitates integration with the Digital Twin for monitoring and optimization.